

Avenant A14a — Le LOD des sphères

LibraryV3 · Phase G, étape 1 · de la Référence I à la Référence J · 4 et 5 octobre 2026

Cet avenant complète l'annexe A14 (chantier LOD des rochers). Il historise, dans l'ordre où elle a été menée, l'étape qui a donné des niveaux de détail aux planètes et aux lunes. Comme A14, il se lit comme une leçon : ce qu'on a fait, pourquoi, ce qui a mal tourné, et ce qu'il faut en retenir. Il ne répète pas les formules (ϵ , q , τ , `SelectLodLevel`) : voir A13, A13 bis et A14 § 1 et § 4.

Résultat en une ligne. Aucune ligne du moteur n'a changé ; trois descripteurs et trois petits `.obj` ont suffi. `FacesSubmitted` passe de 82 988 à 14 752 (−82 %), `Render` de 1,52 à 0,58 ms ($\times 0,38$), la frame de 3,86 à 3,01 ms. La mesure a surtout révélé une loi de coût à deux termes : **`Render` $\approx 0,39 \text{ ms} + 13,5 \text{ ns} \times \text{faces}$** . Le LOD a retiré ce qu'il pouvait ; le poste suivant est `Clear`.

1. Le point de départ et l'exigence

À la Référence I, les rochers étaient en LOD, mais les 40 sphères de la scène (3 en `sphere_hi`, 15 en `sphere_mid`, 22 en `sphere_lo`) restaient en pleine résolution et portaient 88 % des faces encore soumises.

Exigence posée : tout mesh doit pouvoir avoir ou non une chaîne de LOD, car d'autres objets seront ajoutés plus tard. Elle était **déjà satisfaite par le moteur** : depuis l'étape 4d, tout mesh est une chaîne (un `.lod.json`, ou une chaîne implicite de longueur 1 pour un simple `.obj`), et c'est la scène qui décide. Il ne restait qu'une question d'*asset* : comment fabriquer les niveaux.

À retenir. La meilleure preuve qu'une architecture est juste, c'est quand une nouvelle exigence ne demande aucun code. Le choix de l'étape 4d (« un mesh sans LOD est une chaîne de longueur 1, aucun cas particulier ») a payé ici.

2. Examiner avant de mesurer

Les trois sphères sortent de `gen_meshes.py` : `uv_sphere(seg, rings)`, rayon 1, en 16×10, 32×16 et 64×32. On a régénéré les trois fichiers et comparé octet pour octet avec ceux du dépôt : identiques. On sait donc exactement quelle surface on approxime, et on peut fabriquer les niveaux par **ré-échantillonnage exact** de la même fonction, comme pour les rochers.

3. Mesurer ϵ

Méthode des rochers (A14 § 3.2) : écart radial à L0, 100 000 directions aléatoires plus des directions ciblées sur les sommets de tous les niveaux, $\times 1,1$, arrondi vers le haut au dix-millième. On y a ajouté des directions ciblées sur les **centres de faces**.

Niveau	Faces	Sommets moteur	Flèche / sphère exacte	ϵ dans sa chaîne	Admis tant que $R \leq$
hi 64×32	3 968	2 143	0,0024	L0	—
mid 32×16	960	559	0,0096	0,0106	47 px
lo 16×10	288	185	0,0308	0,0330	15 px
12×6	120	89	0,0649	0,0698	7 px
8×5	64	52	0,1213	0,1335	3,7 px
6×4	36	33	0,1848	0,2030	2,5 px

3.1 Deux méthodes indépendantes, un seul résultat

La flèche d'une sphère facettée se calcule aussi analytiquement : pour chaque triangle, distance du centre à son plan, au pied de la perpendiculaire si celui-ci tombe dans le triangle ; la flèche vaut 1 moins le minimum. Les deux méthodes (lancer de rayons échantillonné, géométrie exacte) donnent les mêmes valeurs à 10^{-5} près. Une mesure vérifiée par deux chemins qui ne partagent aucun code est une mesure sûre.

3.2 Où se cache le maximum dépend de la surface

Sur les rochers, le tirage aléatoire sous-estimait ϵ de 46 % : l'écart maximal était un pic étroit près des sommets. Sur les sphères, le tirage aléatoire seul donne déjà le maximum à 10^{-3} près : sur une surface convexe et lisse, l'erreur est une bosse large au centre de chaque face (la flèche), facile à toucher au hasard. La méthode est gardée intacte (elle ne coûte rien), mais la leçon est inverse de celle des rochers.

À retenir. Ce n'est pas la méthode qui décide où se trouve le maximum, c'est la forme de la surface. On cible toujours les endroits où le maximum *peut* se trouver (sommets **et** centres de faces), puis on vérifie la convergence.

3.3 Pour une sphère, ϵ se lit en pixels de rayon

La flèche est proportionnelle au rayon : ϵ ne dépend pas de l'échelle. Le critère $\epsilon_k \times q \leq \tau$, avec q en pixels par unité locale et un rayon local de 1, devient : **le niveau k est admis tant que le rayon apparent $R \leq \tau / \epsilon_k$** . Avec $\tau = 0,5$ px, `sphere_lo` suffit tant que l'astre fait moins de 15 px de rayon à l'écran. La dernière colonne du tableau se lit sans calcul.

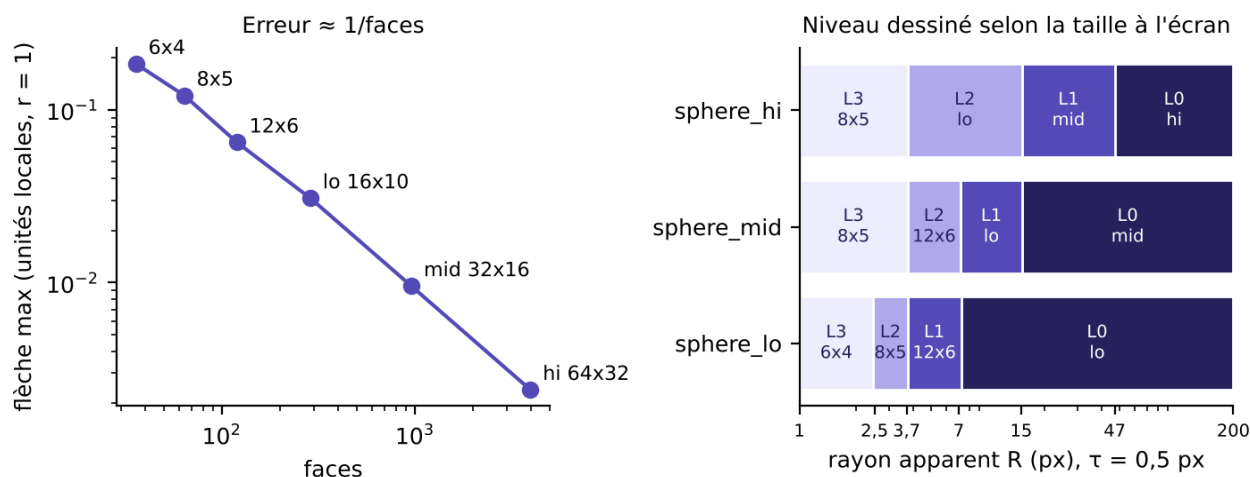


Figure 1 — À gauche, la flèche décroît comme $1/\text{faces}$ (pente -1 en log-log) : diviser l'erreur par 2 coûte 2 fois plus de faces. À droite, le niveau dessiné par chaque chaîne selon le rayon apparent (échelle log). Les trois chaînes partagent les mêmes niveaux bas.

Constat au passage : au-delà de 47 px de rayon, `sphere_mid` dépasse déjà τ au niveau `L0`. Mercure vue de près a une silhouette polygonale visible aujourd'hui, LOD ou pas. Ce n'est pas un défaut du LOD : c'est le choix de résolution de l'auteur de la scène.

4. Décision 1 — Fabriquer les niveaux : deux voies

Les sphères viennent d'une fonction ; un vaisseau modélisé ou un mesh importé, non. Deux voies, déclarées dans le descripteur (clé informative `_fabrication`) :

- "resample" : une fonction génératrice existe (sphères, rochers). On ré-échantillonne la même surface : exact, gratuit, sans artefact.
- "qem" : tout le reste. Simplificateur de Garland-Heckbert (quadriques d'erreur), exécuté hors ligne. Il devient un **sous-chantier séparé**, validé ensuite contre les sphères : QEM(hi) ramené à 960, 288 et 64 faces doit donner $\epsilon_{\text{QEM}} \leq \epsilon_{\text{resample}}$ à nombre de faces égal, couture UV et

pôles intacts.

Les sphères n'avaient pas besoin du QEM : on les a livrées tout de suite, sans attendre l'outil générique.

✗ Un QEM qui ignore la couture UV. La couture d'une sphère UV est une colonne de sommets dupliqués : même position, $u = 0$ d'un côté, $u = 1$ de l'autre (`gen_meshes.py`, correction 1). Un simplificateur naïf voit deux sommets confondus et les fusionne. La géométrie reste juste, ε aussi ; mais le triangle qui enjambe la couture interpole u de 0,98 à 0,02 et balaie toute la texture à l'envers. Une mesure géométrique ne voit pas ce défaut : les arêtes de couture et les rangées polaires doivent être **verrouillées**.

5. Décision 2 — Une chaîne ne monte jamais

Question : une chaîne peut-elle dépasser la résolution choisie par l'auteur ? Mercure, en `sphere_mid`, pourrait passer en `sphere_hi` vue de près, avec une meilleure image.

Décision : non. L0 = la résolution choisie par l'auteur = un plafond. Le LOD est une optimisation : il ne doit jamais changer l'image de plus de τ . Monter, c'est changer l'image, rendre le coût imprévisible ($\times 4$ faces quand la caméra approche) et casser la comparaison « à image identique » avec la référence.

La question disparaît avec **trois descripteurs qui partagent les mêmes .obj** :

Descripteur	L0	L1	L2	L3
<code>sphere_hi.lod.json</code>	hi (3 968)	$\text{mid} \cdot \varepsilon$ 0,0106	$\text{lo} \cdot 0,0330$	$8 \times 5 \cdot 0,1335$
<code>sphere_mid.lod.json</code>	mid (960)	$\text{lo} \cdot 0,0330$	$12 \times 6 \cdot 0,0698$	$8 \times 5 \cdot 0,1335$
<code>sphere_lo.lod.json</code>	lo (288)	$12 \times 6 \cdot 0,0695$	$8 \times 5 \cdot 0,1335$	$6 \times 4 \cdot 0,2030$

Si l'auteur veut Mercure nette de près, il lui donne `sphere_hi.lod.json` ; loin, elle ne coûte rien de plus. **Monter devient une décision d'auteur**, prise dans la scène et mesurable séparément, jamais une décision du moteur.

Les ε d'un même niveau diffèrent à peine d'une chaîne à l'autre ($\text{lo} : 0,0330$ sous `hi` comme sous `mid` ; $12 \times 6 : 0,0698$ sous `mid`, $0,0695$ sous `lo`), parce que `hi` et `mid` sont très proches de la sphère exacte. La règle reste de mesurer **par rapport à L0** : pour un mesh sans fonction génératrice, L0 est la seule référence qui existe.

✗ Le moteur qui « sait mieux » que l'auteur. Une chaîne unique partant de `hi` pour tous les corps rendrait l'image meilleure... et le budget de la frame dépendrait de la position de la caméra. Une optimisation qui change ce qu'on voit n'est plus une optimisation : c'est une décision de contenu, comme le niveau « Culled » écarté en A14 § 9.

6. Les fichiers livrés, et les contrôles appliqués

- `sphere_12x6.obj`, `sphere_8x5.obj`, `sphere_6x4.obj` : générés par la même `uv_sphere`.
- Les trois `.lod.json`, avec `_fabrication`, `_errorMetric` (dont `faceCenterTargeted`) et `_measure` (dont `sagVsExactSphere`) : préfixe « `_` », donc aucun avertissement `WarnUnread`.
- `gen_sphere_lod.py`, pour la traçabilité (il importe `gen_meshes.py` et `gen_rock_lod.py`).
- Une scène `solar_system_v1compat_belt_lod_spheres.json` : copie de la scène de la Référence I où seuls les 40 chemins `model` changent. La scène de référence reste intacte, et c'est la comparaison des deux qui mesure le LOD (même principe qu'en A14 § 7.5).

Deux leçons d'A14 appliquées d'emblée : les sommets annoncés sont ceux que compte le moteur, c'est-à-dire les triplets référencés par les faces (559 et non 561 pour `mid` : A14 § 3.3) ; et les clés de documentation portent le préfixe « `_` » (A14 § 9).

Le premier chargement s'est fait sans avertissement : trois chaînes de 4 niveaux, aucun LevelMismatch.

7. La mesure

7.1 Un premier run, des compteurs justes et des temps irrecevables

Le premier run envoyé portait dans son en-tête `affinity=fff` : pas d'épinglage sur un P-core (M9), un run isolé sans ABBA (M8), un témoin Clear à +16 % de la Référence I. Ses **temps** ont été refusés. Ses **compteurs**, eux, sont recevables : la simulation est déterministe, ils ne dépendent ni de la machine ni de la séance. L'invariant $\Sigma \text{LodLevel} = \text{MeshesTested} - \text{MeshesCulled}$ a tenu sur les 240 frames.

Ce run a quand même servi : la corrélation frame par frame entre Render et FacesSubmitted y valait **0,04**. Les faces n'expliquaient plus Render. D'où l'hypothèse d'un coût fixe, et la forme de la règle écrite ensuite.

À retenir. On lit l'en-tête avant le premier chiffre (M13). Un run irrecevable pour les temps peut rester recevable pour les compteurs : il faut savoir quelle grandeur dépend de l'état de la machine, et laquelle n'en dépend pas.

7.2 RelWithAsserts ou Release ?

Deux configurations, deux questions. **RelWithAsserts** dit si c'est juste (assertions, invariants) ; ses compteurs doivent être identiques frame par frame à ceux de Release, sinon une assertion a un effet de bord. **Release** dit combien ça coûte. On ne mélange jamais les deux dans un lot. Le lot de temps a été mené en Release (`asserts_lib=0` dans les cinq en-têtes).

7.3 Le protocole : un seul binaire

A et B ne diffèrent que par la scène : **un seul exécutable** sert aux deux, ce qui supprime tout risque d'étiquetage (A14 § 8.2). Lot M13 : i7-1265U sur secteur, `start "" /wait /affinity 4`, un run jetable, puis A1 B1 B2 A2.

7.4 La règle, écrite avant la mesure (M12)

- **Recevabilité** : `affinity=4` partout ; témoins T et Clear à moins de 3 % dans chaque paire ; $\text{Render p95} \div \text{médiane} < 1,15$ ou $\text{p95} - \text{médiane} < 0,3 \text{ ms}$; compteurs $A1 = A2$ et $B1 = B2$ frame par frame.
- **Modèle** : $\text{Render} = a + b \times F$. b se calcule frame par frame, jamais par différence de médianes.
- **Zones de B ÷ A** : $\leq 0,25$, la loi proportionnelle tient ; 0,25 à 0,60, coût fixe confirmé ; $> 0,60$, anomalie.
- **Prédiction** : zone 0,25 à 0,60 ; b entre 15 et 25 ns par face.

7.5 Recevabilité

Contrôle	Résultat
En-têtes	Release, <code>asserts_lib=0</code> , <code>affinity=4</code> sur les 5 runs : conforme
Compteurs $A1 = A2$, $B1 = B2$	identiques sur les 240 frames ; même état de simulation en A et B à chaque frame
Témoin T (calcul)	+2,2 % (paire 1), +2,7 % (paire 2) : conforme
Témoin Clear (mémoire)	+5,4 % (paire 1) : hors règle ; -1,3 % (paire 2) : conforme
Dispersion de Render	$\text{p95} \div \text{médiane}$ jusqu'à 1,35, mais $\text{p95} - \text{médiane} \leq 0,28 \text{ ms}$ partout : recevable par le critère amendé

La paire 1 est conservée **sous réserve déclarée**, sans réécrire la règle après coup. Elle ne porte pas le verdict : la paire 2, recevable sans réserve, donne le même rapport à 1,6 % près.

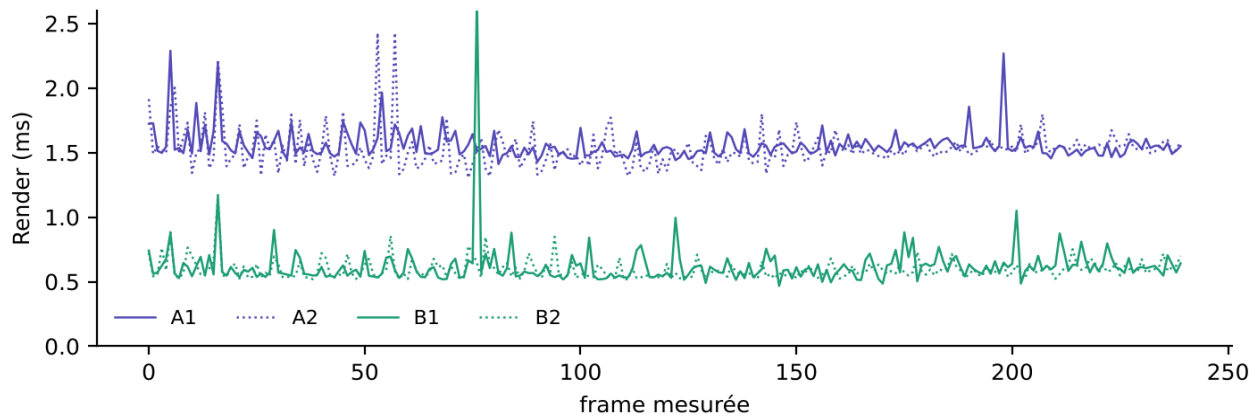


Figure 2 — Render frame par frame. Les deux A se superposent, les deux B aussi : seul le LOD des sphères sépare les deux familles.

7.6 Résultats

	A (Référence I)	B (sphères en LOD)	B ÷ A
Render, paire 1	1,532 ms	0,590 ms	0,385
Render, paire 2	1,510 ms	0,573 ms	0,379
Frame, paire 1	3,85 ms	3,07 ms	0,80
Frame, paire 2	3,86 ms	2,96 ms	0,77
FacesSubmitted	82 988	14 752	0,178
LodLevel0 / LodLevel3	97 / 1 200	12 / 1 285	—

Les 97 paires en L0 de A sont les 85 sphères visibles plus 12 autres objets (chaînes implicites). En B, les 85 sphères sont toutes en L3 : vues par quatre caméras en 1536 × 900, aucune ne dépasse 3,7 px de rayon. L1 et L2 ne servent jamais dans cette scène, comme pour les rochers.

8. La loi de coût, et une prédiction réfutée

La frame i de A et la frame i de B ont exactement le même état de simulation ; leur seule différence est le LOD. Pour chaque frame :

```
b_i = (Render_A[i] - Render_B[i]) / (Faces_A[i] - Faces_B[i]) puis b = médiane des b_i
a = Render_B - b × Faces_B
```

Résultat : **b = 13,5 ns par face** (13,6 et 13,4 selon la paire, écart interquartile de 12,3 à 14,5) et **a ≈ 0,39 ms**. Vérification : $0,39 + 13,5 \text{ ns} \times 82\,988 = 1,51 \text{ ms}$ (mesuré 1,51 à 1,53) ; $0,39 + 13,5 \text{ ns} \times 14\,752 = 0,59 \text{ ms}$ (mesuré 0,57 à 0,59). Le rapport B ÷ A tombe dans la zone 0,25 à 0,60 : coût fixe confirmé.

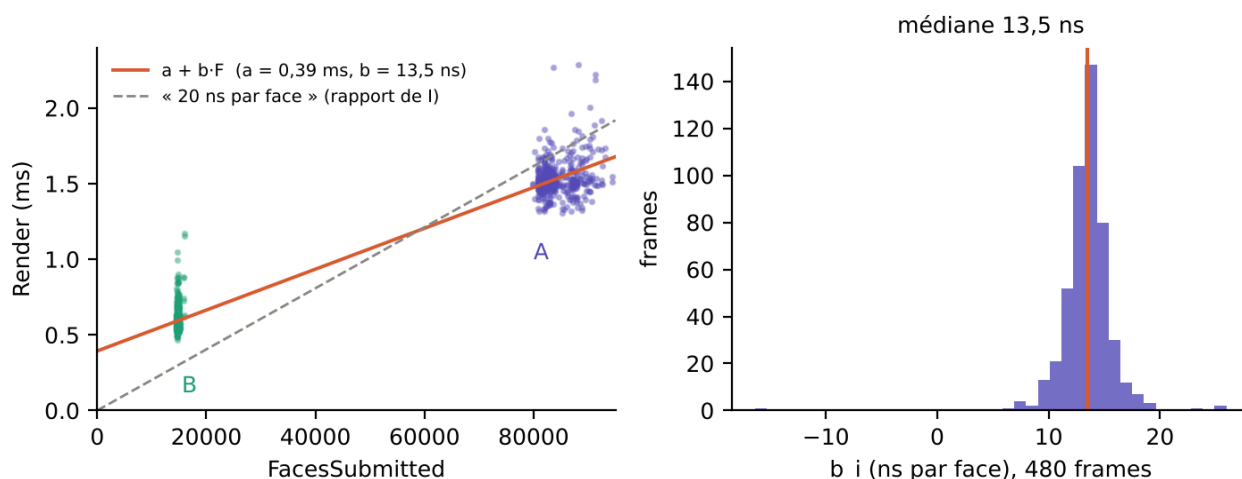


Figure 3 — À gauche, les 960 frames : la droite $a + b \cdot F$ passe par les deux nuages ; la droite pointillée (« 20 ns par face », passant par l'origine) prédisait B à 0,30 ms et se trompe d'un facteur 2. À droite, la distribution des 480 pentes b_i : étroite, centrée sur 13,5 ns.

La prédiction sur b (15 à 25 ns) est réfutée. Elle partait des « 20 ns par face » de la Référence I. Or ce chiffre était un **rapport** (Render ÷ faces), pas une **pen**te. Les deux ne coïncident que si le coût fixe est nul :

- Référence I : $1,68 \text{ ms} \div 82\,988 = 20,2 \text{ ns}$, dont 13,5 ns de pente et 6,7 ns de coût fixe réparti sur les faces ;
- Référence J : $0,58 \text{ ms} \div 14\,752 = 39 \text{ ns par face}$, alors que la pente n'a pas changé.

Le coût fixe existait déjà à la Référence I ($\approx 26\%$ de Render) ; il en fait maintenant les deux tiers. Il ne dépend pas du nombre de faces : 2 580 tests de culling, autant de matrices MVP et de sélections de niveau, puis la préparation de chaque instance dessinée.

✗ Extrapoler un coût unitaire mesuré en un seul point. Un « coût par face » calculé comme $\text{Render} \div \text{faces}$ mélange ce qui est fixe et ce qui est proportionnel. Il prédit faux dès que le nombre de faces change beaucoup. Il faut au moins deux points, sur la même machine et dans la même séance, pour séparer a et b. Ici, c'est le LOD lui-même qui a fourni le second point.

9. La Référence J, et ce qu'elle décide

Référence J	Valeur
Machine	i7-1265U, P-core épinglé (affinity=4), secteur
Scène	solar_system_v1compat_belt_lod_spheres.json, 4 vues, 1536 × 900, Release
Render	0,57 à 0,59 ms ($\approx 19\%$ de la frame)
Frame	2,96 à 3,07 ms (≈ 330 fps)
Clear	1,25 à 1,28 ms ($\approx 42\%$)
Present	0,94 à 0,99 ms ($\approx 32\%$, hors budget moteur)
FacesSubmitted	14 752
Loi de coût	$\text{Render} \approx 0,39 \text{ ms} + 13,5 \text{ ns} \times \text{FacesSubmitted}$

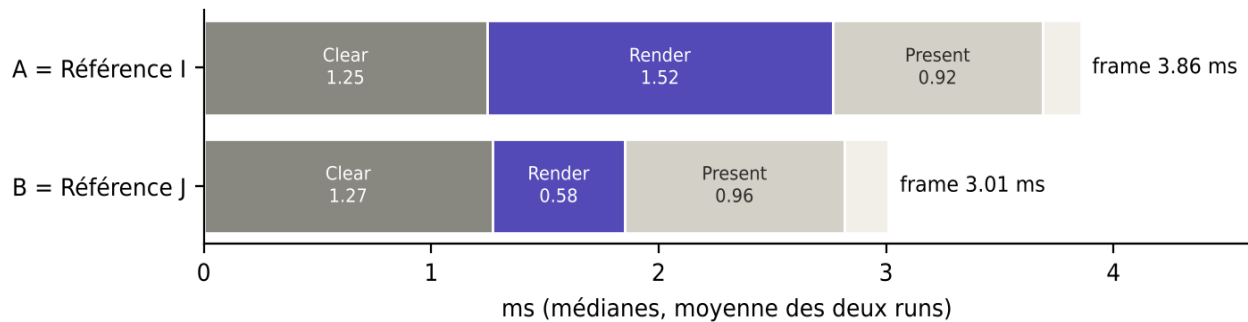


Figure 4 — Répartition de la frame. Render a fondu ; Clear n'a pas bougé et pèse désormais plus du double de Render.

Les faces restantes ne coûtent plus que 0,20 ms : même un LOD parfait ne gagnerait pas davantage dans cette scène. **Le chantier LOD a fait son travail.** Le premier poste du moteur est maintenant Clear, limité par la mémoire et indépendant des faces : c'est l'étape 2 du plan, désormais justifiée par la mesure.

10. Les contre-exemples, rassemblés

- **X** Supposer que le maximum d'erreur se trouve toujours au même endroit : aux sommets sur un rocher, au centre des faces sur une sphère (§ 3.2).
- **X** Simplifier sans verrouiller la couture UV et les pôles : géométrie juste, texture balayée à l'envers (§ 4).
- **X** Laisser le moteur monter au-dessus du choix de l'auteur : image changée, budget imprévisible (§ 5).
- **X** Lire les temps d'un run sans lire son en-tête : affinity=fff (§ 7.1).
- **X** Réécrire une règle de recevabilité après avoir vu les chiffres : on déclare la réserve à la place (§ 7.5).
- **X** Prendre un rapport pour une pente : « 20 ns par face » n'était pas le coût marginal (§ 8).

11. Ouvert, dettes et nettoyage

Point	Où	Quand
Sous-chantier QEM (couture et pôles verrouillés, validé contre les sphères)	outil hors ligne	à l'arrivée d'un mesh sans fonction génératrice
Décomposer a $\approx$ 0,39 ms (par mesh testé ou par instance dessinée), par ablation	RenderView	plus tard, non prioritaire
Run RelWithAsserts de la scène des sphères : non versé au dossier de mesure	banc	à archiver s'il a été fait
Supprimer assets/Meshesrock_b.lod.json et assets/Meshesrock_c.lod.json (copies périmées)	dépôt EXE	prochain commit
Aligner gen_rock_lod.py sur les clés « _errorMetric », « _measure »	générateur	prochain commit
Dettes Serializer et ResourceManager (A14 § 11)	LIB	étape 3 du plan

12. Aide-mémoire

Grandeur	Valeur ou règle
Flèche d'une sphère UV	$\approx 1/\text{faces}$; vérifiable analytiquement (distance au plan de chaque triangle)
Seuil d'un niveau de sphère	$R \leq \tau / \varepsilon_k$ pixels de rayon (rayon local 1)
Fabrication	resample si fonction génératrice, sinon qem (couture verrouillée)
Droit de monter	non : L0 = plafond choisi par l'auteur, un descripteur par plafond
Loi de coût de Render (J)	$0,39 \text{ ms} + 13,5 \text{ ns} \times \text{FacesSubmitted}$
Calcul d'une pente	frame par frame entre deux scènes au même état de simulation, puis médiane
Prochaine étape	Clear ($\approx 42\%$ de la frame)